

TABLE DES MATIERES

1. Resume Executif
2. Etat des Lieux : Code vs Documentation
3. Analyse des EcartS par Severite
4. Phase 0 : Fondations Architecturales (S1-S6)
5. Phase 1 : Auth, RBAC et Securite (S7-S12)
6. Phase 2 : Mode Offline et POS (S13-S20)
7. Phase 3 : Modules Metier Core M01-M09 (S21-S32)
8. Phase 4 : Espace Patient et Analytics IA (S33-S38)
9. Phase 5 : Modules Institutionnels M16-M19 (S39-S44)
10. Phase 6 : Integrations Grossistes et SoBAPS (S45-S48)
11. Phase 7 : Reseau Network et Deploiement (S49-S52)
12. Phase 8 : Certification et Mise en Production (S53-S56)
13. Schema Prisma : Changements Requis
14. Variables d'Environnement Manquantes
15. Risques et Mitigation
16. Calendrier Consolidé

1. RESUME EXECUTIF

Le projet MediHelm, tel que documente dans les fichiers CONTEXT.md, medihelm.md et medihelm.cursorrules, decrit une infrastructure pharmaceutique nationale SaaS multitenant de grande ambition : 19 modules fonctionnels, architecture monorepo, backend NestJS, mode offline critique, alertes DPMED en moins de 2 minutes, et integrations institutionnelles avec la DPMED, la SoBAPS, l'ABRP et les grossistes repartiteurs. Cependant, l'analyse approfondie du codebase actuel revele un ecart considerable entre la documentation et la realite du code.

Le projet actuel est une application Next.js 16 monolithique sans monorepo, sans backend NestJS separe, sans mode offline fonctionnel, sans systeme de files d'attente BullMQ/Redis, et avec seulement 6 routes API sur 112 qui verifient l'authentification. Les integrations Fedapay et AfricasTalking sont simulees, le RBAC existe dans le code mais n'est applique que sur 5% des routes, et les validations d'entrees (Zod) sont totalement absentes malgre la presence du package. Le schema Prisma contient 77 modeles mais diverge sur plusieurs champs critiques par rapport a la documentation (Vente.reference, Vente.synchedAt, Pharmacie.slug, DomaineIA, etc.).

Ce document propose un plan d'action en 8 phases s'etaland sur 56 semaines pour atteindre une conformite complete avec la documentation. Les phases sont ordonnees par priorite : d'abord les fondations architecturales et la securite, puis les modules metier, et enfin les integrations institutionnelles. Chaque phase inclut des livrables concrets, des criteres d'acceptation, et des dependances explicites. L'objectif est de transformer le prototype actuel en une plateforme de production robuste, securisee et conforme aux specifications MH-CDC-2025-v2.0.

Indicateur	Etat Actuel	Objectif Final
Architecture	Monolithe Next.js 16	Monorepo (apps/api + apps/web + packages/)
Backend	Next.js API routes (112 fichiers)	NestJS + Fastify (20 modules)
Authentification	NextAuth v4 (JWT sans verification signature)	Supabase Auth + JWT (15min/7j)
Routes securisees	6/112 (5%)	112/112 (100%)
Validation entrees	0/112 (0%)	112/112 (100% Zod)
RBAC applique	5% des routes	100% des routes
Mode offline	Service Worker basique (assets statiques)	SQLite + sync + next-pwa complet
Queues/Redis	Aucun	BullMQ + Redis Upstash
WebSocket	Exemple non integre	NestJS Gateway + rooms pharmacie

Indicateur	Etat Actuel	Objectif Final
Integrations paiement/SMS	Simulees (Fedapay, AfricasTalking)	Reelles avec clefs chiffrees

2. ETAT DES LIEUX : CODE vs DOCUMENTATION

2.1 Architecture

La documentation specifie une architecture monorepo avec trois niveaux : apps/api (NestJS), apps/web (Next.js), et packages/ (shared-types, ui, utils). Le code actuel est un projet Next.js unique et plat. Tout le backend est implemente via des API routes dans src/app/api/, sans separation des responsabilites. Il n'existe aucune structure de monorepo, aucun package partage, et aucun backend dedie. Cette architecture monolithique rend le code difficile a tester, a deployer independamment, et a faire evoluer selon les principes de la documentation.

2.2 Stack Technique

Composant	Documente	Actuel	Ecart
Frontend	Next.js 14 App Router	Next.js 16 App Router	Version superieure (acceptable)
Backend	NestJS + Fastify	Next.js API routes	CRITIQUE : absence totale
ORM	Prisma 5+	Prisma 6.11	Version superieure (acceptable)
Base de donnees	PostgreSQL 15 Supabase + RLS	PostgreSQL (Neon) sans RLS	RLS non active
Auth	Supabase Auth + JWT	NextAuth v4 + JWT sans verification	CRITIQUE : fournisseur different
Cache offline	SQLite via Prisma	Aucun	CRITIQUE : absent
Cache serveur	Redis Upstash	Aucun	Absent
Queue	BullMQ	Aucun	CRITIQUE : absent
PWA	next-pwa	SW basique (assets statiques)	CRITIQUE : non fonctionnel
SMS	AfricasTalking	Simule	Non integre
Paieement	Fedapay SDK	Simule (src/lib/fedapay.ts)	Non integre
WebSocket	NestJS Gateway	Exemple non connecte + SSE	Non integre
Monitoring	Sentry + Vercel Analytics	Aucun	Absent

2.3 Securite

L'analyse de securite revele des vulnerabilites majeures. Seulement 6 des 112 routes API verifient l'authentification, et aucune ne valide les entrees avec Zod. Le systeme RBAC, bien que complet dans rbac.ts, n'est applique que sur 5% des routes. Le bearer token JWT est decode sans verification de signature (utilisation de atob()), ce qui permet potentiellement de forger des tokens. Il n'y a aucun rate limiting, aucun mTLS, et les identifiants de base de donnee sont en clair dans les scripts (scripts/update-pharmacies-osm.ts). Les webhooks HMAC-SHA256 sont correctement implements, mais c'est la seule mesure de securite significative en place.

2.4 Schema Prisma

Le schema Prisma contient 77 modeles et 31 enums, ce qui couvre l'ensemble des 19 modules documentes. Cependant, plusieurs champs critiques sont manquants ou divergents. Le modele Vente manque de reference (unique), synchedAt, et montantAssur. Le modele Pharmacie manque de slug, modeGardeActif et planExpireAt. L'enum DomaineIA n'existe pas (remplace par un champ String). L'enum PlanTarifaire utilise SEED au lieu de STEM, et l'enum StatutVente n'inclut pas AVOIR. Ces divergences doivent etre corrigees pour garantir la conformite avec la documentation et le bon fonctionnement du mode offline et du systeme de pricing.

3. ANALYSE DES ECARTS PAR SEVERITE

3.1 Ecart Critique (Bloquant pour la production)

Absence NestJS

Tout le backend est dans Next.js API routes. Impossible de separer les responsabilites, d'appliquer les guards/interceptors/middleware NestJS, ou de deployer independamment.

95% routes sans auth

106 routes API sur 112 n'ont aucune verification d'authentification. N'importe qui peut acceder aux donnees de n'importe quelle pharmacie.

JWT sans verification

Le bearer token est decode avec atob() sans verifier la signature. Un attaquant peut forger un token avec n'importe quel payload.

Zero validation d'entrees

Aucune route ne valide les entrees avec Zod. Risques d'injection, de corruption de donnees, et de comportements imprevisibles.

Pas de mode offline

Le POS ne peut pas fonctionner hors connexion. Or, au Benin, les coupures internet et electriques sont frequentes. C'est un bloqueur absolu.

Pas de BullMQ/Redis

Les alertes DPMED ne peuvent pas etre diffusees en moins de 2 minutes sans systeme de queue dedie. Le SLA critique est impossible a respecter.

Pas de WebSocket reel

Pas de notifications temps reel. Les commandes patient, les alertes stock et les alertes DPMED ne peuvent pas etre poussees instantanement.

3.2 Ecart Elevé (Impact majeur sur la fiabilite)

RBAC non applique

Le systeme RBAC est complet dans rbac.ts mais n'est utilise que sur 6 routes. Un caissier peut acceder au module finance, un magasinier peut voir les ventes.

Pas de rate limiting

Aucune protection contre les attaques par force brute sur /auth/login et les autres endpoints sensibles.

Fedapay simule

Les paiements en ligne sont simulés. Aucune transaction réelle n'est possible.

AfricasTalking simule

Les SMS ne sont pas envoyés. Les alertes DPMED par SMS sont impossibles.

Pas de mTLS

Les connexions aux institutions (DPMED, SoBAPS) ne sont pas sécurisées par mTLS.

Clefs API en clair

Les identifiants Neon DB sont en clair dans les scripts source. Aucun chiffrement des clefs API en base.

Pas de Sentry

Aucun monitoring d'erreurs en production. Les incidents resteront invisibles.

3.3 Ecart Moyens (Fonctionnalités incomplètes)

Champs Prisma manquants

Vente.reference, Vente.synchedAt, Vente.montantAssur, Pharmacie.slug, Pharmacie.modeGardeActif, Pharmacie.planExpireAt, DomaineIA enum.

Enums divergents

PlanType(SEED vs STEM), StatutVente(sans AVOIR), ModePaiement(sans ASSURANCE/CREDIT), StatutCommandePatient(en anglais).

Pas de FEFO

Le décrement stock FEFO n'est pas implémenté dans les routes de vente.

Pas de CMUP

Le calcul du Coût Moyen Unitaire Pondéré n'est pas automatisé.

Pas de cron Analytics

Les rapports Analytics IA ne sont pas générés automatiquement à 05h00 WAT.

Portails institutionnels incomplets

Les pages DPMED, SoBAPS, ABRP existent mais les flux métier réels ne sont pas connectés.

Pas de Docker/CI-CD

Aucun Dockerfile, aucun pipeline CI/CD, aucun script de déploiement.

PHASE 0 : FONDATIONS ARCHITECTURALES

Durée estimée : Semaines 1-6 (6 semaines)

Cette phase est la base de tout. Elle transforme le monolithe Next.js en monorepo avec un backend NestJS separé et des packages partagés. Sans cette restructuration, aucune des phases suivantes ne peut être exécutée correctement, car les guards, middleware, interceptors et WebSocket Gateway de NestJS sont indispensables pour la sécurité, le multitenancy et le temps réel.

4.1 Livrables

- Structure monorepo fonctionnelle : apps/api/, apps/web/, packages/shared-types/, packages/ui/, packages/utils/
- Configuration Turborepo ou Nx avec workspace dependencies
- Backend NestJS + Fastify opérationnel avec health check et connexion Prisma
- Frontend Next.js 16 migre dans apps/web/ avec toutes les pages existantes
- Package shared-types avec tous les enums et interfaces partagés (RoleType, PlanType, StatutVente, etc.)
- Package ui avec les composants React partagés extraits de src/components/ui/
- Package utils avec les fonctions utilitaires partagés de src/lib/
- ESLint + Prettier configurés à l'échelle du monorepo
- Scripts de build et dev fonctionnels pour les deux applications
- Migration du schema Prisma à la racine du monorepo avec les corrections de champs manquants

4.2 Etapes détaillées

Semaine	Action
S1	Initialiser le monorepo avec pnpm workspaces ou Turborepo. Créer la structure de dossiers apps/ et packages/.
S1-S2	Créer apps/api/ avec NestJS + Fastify. Configurer le module Prisma, le module Config, et le health check.
S2-S3	Migrer le frontend Next.js dans apps/web/. Mettre à jour tous les imports relatifs. Vérifier que l'app démarre.
S3-S4	Créer packages/shared-types/ : extraire tous les enums Prisma et interfaces TypeScript des deux apps.
S4	Créer packages/ui/ : extraire les composants partagés (Button, Input, Table, Modal, etc.).
S4-S5	Créer packages/utils/ : extraire les fonctions utilitaires (formatCurrency, formatDate, etc.).

Semaine	Action
S5-S6	Mettre a jour le schema Prisma avec les champs manquants (Vente.reference, synchedAt, montantAssur, etc.). Generer la migration.
S6	Tests de bout en bout : les deux apps demarrent, les packages sont importables, le build passe.

4.3 Criteres d'acceptation

- pnpm install && pnpm build passe sans erreur
- apps/api/ demarre sur le port 3001 avec health check /health renvoyant 200
- apps/web/ demarre sur le port 3000 avec toutes les pages accessibles
- packages/shared-types est importable dans les deux apps
- La migration Prisma s'applique sans erreur sur la base de donnees

PHASE 1 : AUTH, RBAC ET SECURITE

Durée estimée : Semaines 7-12 (6 semaines)

La securite est la priorite absolue. Avec seulement 5% des routes securisees et un JWT decode sans verification de signature, la plateforme est vulnérable a des attaques triviales. Cette phase met en place l'authentification Supabase Auth, applique le RBAC sur toutes les routes, ajoute la validation Zod systematique, et implemente le rate limiting et le chiffrement des clefs API.

5.1 Livrables

- Authentification Supabase Auth integree avec JWT (access 15 min, refresh 7 jours)
- NestJS AuthGuard applique sur 100% des endpoints proteges
- NestJS RolesGuard avec decorateur @Roles() sur tous les endpoints selon le RBAC
- TenantMiddleware injectant SET app.current_tenant avant chaque requete Prisma
- Validation Zod sur 100% des endpoints avec pipes NestJS (ZodValidationPipe)
- Rate limiting sur /auth/login : 5 tentatives / 15 min / IP
- Rate limiting global : 100 req/min par IP sur les autres endpoints
- Chiffrement des clefs API en base (AES-256-GCM) avec cle maitresse dans env
- Suppression des identifiants en clair dans le code source
- Audit logging systematique via le modele AuditLog existant

5.2 Etapes detaillees

Semaine	Action
S7	Installer et configurer @supabase/supabase-js et @supabase/auth-helpers-nextjs. Creer le module NestJS AuthModule.
S7-S8	Implementer AuthGuard NestJS avec verification JWT Supabase. Remplacer le decodeBearerToken() vulnérable.
S8-S9	Implementer RolesGuard avec decorateur @Roles(). Mapper les 12 roles du RBAC. Appliquer sur tous les endpoints.
S9	Implementer TenantMiddleware : injection SET app.current_tenant. Ajouter filtre pharmacieId sur toutes les queries Prisma du backend.
S9-S10	Creer ZodValidationPipe NestJS. Ecrire les schemas Zod pour chaque endpoint (body, query, params). Appliquer systematiquement.
S10	Implementer le rate limiting avec @nestjs/throttler ou un middleware Redis. Configurer les limites par endpoint.
S10-S11	Implementer le chiffrement des clefs API (AES-256-GCM). Migrer les clefs existantes. Nettoyer le code source.

Semaine	Action
S11-S12	Implementer l'audit logging systematique. Tester la conformite RBAC avec des tests d'integration par role.

5.3 Criteres d'acceptation

- 100% des endpoints proteges requierent un JWT valide Supabase
- Chaque role ne peut acceder qu'aux modules autorises par le RBAC
- Toute requete Prisma backend filtre par pharmacieId automatiquement
- Tous les inputs sont valides par Zod avant traitement
- Rate limiting actif sur /auth/login (5/15min) et endpoints generaux (100/min)
- Aucune cle API en clair dans le code source ou la base de donnees
- AuditLog enregistre chaque action sensible

PHASE 2 : MODE OFFLINE ET POS

Durée estimée : Semaines 13-20 (8 semaines)

Le mode offline est qualifié de CRITIQUE dans la documentation. Au Bénin, la connexion internet est instable et les coupures électriques fréquentes. Le POS doit fonctionner hors connexion, les ventes doivent être enregistrées localement dans SQLite et synchronisées automatiquement au retour du réseau. Aucune vente ne doit être perdue, même après une heure de coupure. Cette phase implémente le cache SQLite via Prisma, la synchronisation offline/online, et la PWA complète avec next-pwa.

6.1 Livrables

- SQLite local via Prisma pour les tables critiques (médicaments, lots, patients en lecture ; ventes, lignes_vente, paiements en écriture)
- Service de synchronisation offline/online avec file d'attente locale et résolution de conflits
- Endpoint POST /ventes/sync pour la synchronisation batch idempotente
- Champ syncedAt sur Vente et LigneVente : null si créé offline, date si synchronisé
- Champ référence unique sur Vente générée localement (UUID ou horodatage) pour la idempotence
- Indicateur visuel permanent de l'état de connexion dans l'interface POS
- PWA complète avec next-pwa : service worker avancé, manifest.json, installation sur Android
- Survie aux redémarrages : les données SQLite persistent même si le navigateur est fermé
- Tests de résistance : 100 ventes créées offline, synchronisées sans perte au retour réseau

6.2 Architecture offline proposée

L'architecture offline repose sur trois couches. La première est la base SQLite locale (via better-sqlite3 ou sql.js) qui stocke les médicaments, lots et patients pour la lecture rapide, et les ventes/lignes/paiements pour l'écriture locale. La deuxième couche est la file d'attente de synchronisation qui stocke les opérations en attente dans IndexedDB (pour la persistance navigateur) et les envoie au serveur quand le réseau revient. La troisième couche est le service worker qui intercepte les requêtes API, les met en cache quand le réseau est disponible, et sert les données en cache quand il est hors ligne.

6.3 Étapes détaillées

Semaine	Action
S13-S14	Implementer la base SQLite locale avec Prisma. Creer le schema offline et les repositories de lecture/écriture.
S14-S15	Implementer le SyncService : detection offline/online, file d'attente IndexedDB, retry automatique.
S15-S16	Implementer l'endpoint POST /ventes/sync (NestJS). Logique idempotente basee sur Vente.reference unique.
S16-S17	Integrer next-pwa dans apps/web/. Configurer le service worker pour le cache strategique.
S17-S18	Implementer l'indicateur visuel de connexion (hook useOnlineStatus + composant OfflineBanner).
S18-S19	Implementer le decrement FEFO dans le POS offline : tri des lots par dateExpiration asc.
S19-S20	Tests de resistance : scenario de 100 ventes offline, reprise reseau, verification zero perte. Tests coupure électrique.

PHASE 3 : MODULES METIER CORE M01-M09

Durée estimée : Semaines 21-32 (12 semaines)

Cette phase implemente les modules metier fondamentaux dans le backend NestJS. Chaque module correspond a un module documente (M01 a M09) avec ses services, controleurs, DTOs, et tests. Les routes API Next.js existantes sont migrees vers les endpoints NestJS correspondants. Le frontend est mis a jour pour appeler les nouvelles routes. L'accent est mis sur le respect des regles metier documentees : FEFO pour le stock, CMUP pour les prix, multitenancy strict, et RBAC par module.

7.1 Modules et priorites

Module	Nom	Semaines	Dependances	Points critiques
M01	Gestion du Stock	S21-S23	Phase 0+1	FEFO, CMUP, alertes peremption, stock minimum/securite
M02	Point de Vente (POS)	S23-S26	M01 + Phase 2	Multi-caissier, offline, ordonnances, reçus PDF
M03	Commandes Fournisseurs	S26-S28	M01, M04	Bons de commande, reception, retours, API grossiste
M04	Gestion Fournisseurs	S28-S29	Phase 0+1	Referentiel, conditions, score fiabilite
M05	Gestion Patients	S29-S30	Phase 0+1	Dossier, historique, fidelite, credit
M06	Ordonnances	S30-S31	M01, M05	Numerisation, validation, stupefiants, interactions
M07	Personnel (RH)	S31-S32	Phase 0+1	Planning, congés, pointage, paie CNSS/IRPP Benin
M08	Finance	S31-S32	M02	Caisse journaliere, resultat, TVA, export SYSCOHADA
M09	Pharmacie de Garde	S32	M01, M05	Planning, diffusion, rapport, alertes patients

7.2 Regles metier a implementer

- FEFO (First Expired, First Out) : lors de chaque vente, les lots sont consommés dans l'ordre croissant de dateExpiration. Le pharmacien peut forcer un lot manuellement.
- CMUP (Cout Moyen Unitaire Pondere) : recalcule automatiquement a chaque reception de lot. Formule : $(\text{valeur stock existant} + \text{valeur reception}) / (\text{quantite existante} + \text{quantite recue})$.

- Multitenancy strict : chaque query Prisma filtre par pharmacieId via le TenantMiddleware. Aucune requete sans filtre.
- RBAC par module : chaque endpoint est protege par AuthGuard + RolesGuard avec les roles autorises selon la matrice RBAC.
- Devise FCFA (XOF) : aucun arrondi automatique, pas de conversion. Tous les montants en entiers ou avec 2 decimales max.
- Fuseau WAT (UTC+1) : tous les timestamps en WAT. Les dates sont affichees et stockees en UTC+1.
- CNSS Benin : part salariale 3,6%, part patronale 15,4%. IRPP par tranches (0%, 10%, 15%, 19%, 28%).
- SYSCOHADA revise : plan comptable ouest-africain, pas le plan francais.

7.3 Migration des routes API

Chaque module NestJS suit le meme pattern : un module (NestModule), un controleur (Controller) avec les endpoints REST documentes, un service (Service) avec la logique metier, des DTOs avec validation Zod, et des tests unitaires et d'integration. Les routes API Next.js existantes sont progressivement remplacees par des appels au backend NestJS. Le frontend est mis a jour pour utiliser NEXT_PUBLIC_API_URL comme base URL pour les appels API. Pendant la transition, les deux systemes cohabitent avec un feature flag par module.

PHASE 4 : ESPACE PATIENT ET ANALYTICS IA

Durée estimée : Semaines 33-38 (6 semaines)

Cette phase couvre l'espace patient (route (patient) dans la documentation) et le module Analytics IA (M15). L'espace patient est l'interface grand public qui permet aux patients de rechercher des médicaments, trouver les pharmacies de garde, passer des commandes en ligne, et gérer leur dossier santé. Le module Analytics IA génère des rapports quotidiens avec des scores de santé par domaine et des prédictions de rupture de stock.

8.1 Espace Patient

- Recherche de médicaments : GET /public/medicaments/search avec recherche full-text sur DCI et nom commercial
- Pharmacies proches : GET /public/gardes avec geolocalisation via latitude/longitude et filtres de garde
- Commande en ligne : POST /public/commandes avec numéro de passage et notification WebSocket à la pharmacie
- Compte patient : inscription, connexion, historique des commandes et ordonnances
- Fidelite : points de fidelite, historique des transactions, seuils de remise
- Paiement Fedapay : intégration réelle avec Wave, MTN Money, Moov Money et carte bancaire
- Commission 1,5% plafonnée à 500 FCFA par commande sur les paiements en ligne

8.2 Analytics IA (M15)

Le module Analytics IA génère des rapports quotidiens à 05h00 WAT via un cron BullMQ. Chaque rapport couvre 9 domaines (STOCK, VENTES, PATIENTELE, PERSONNEL, FINANCE, PEREMPTIONS, RESEAU, CONFORMITE, INTEGRATION_GROSSISTE) avec un score de 0 à 100 points. Les scores sont codés par couleur : vert (75+), ambre (50-74), rouge (moins de 50). L'algorithme de prédiction de rupture calcule la consommation moyenne journalière sur 30 jours et estime les jours avant rupture. L'endpoint POST /analytics/relancer permet de régénérer manuellement un rapport avec un cooldown de 30 minutes.

8.3 Etapes détaillées

Semaine	Action
S33-S34	Implementer les endpoints publics (medicaments/search, gardes, commandes) dans NestJS avec rate limiting.
S34-S35	Implementer le compte patient complet avec Supabase Auth. Migrer les pages (patient) existantes.

Semaine	Action
S35-S36	Integrer Fedapay SDK reel. Implementer les paiements Wave, MTN, Moov, carte. Webhook de confirmation.
S36-S37	Implementer le module Analytics IA avec les 9 domaines. Cron quotidien via BullMQ. Algorithme prediction.
S37-S38	Implementer l'endpoint /analytics/relancer avec cooldown. Dashboard Analytics dans le frontend. Tests.

PHASE 5 : MODULES INSTITUTIONNELS M16-M19

Durée estimée : Semaines 39-44 (6 semaines)

Les modules M16 a M19 sont les différenciateurs absolus de MediHelm face à la concurrence. Ils transforment un logiciel de gestion de pharmacie en une infrastructure sanitaire nationale. Le module M16 (Pharmacovigilance) permet le signalement d'effets indésirables et la veille qualité. Le module M18 (Alertes DPMED) est le canal officiel de diffusion nationale avec un SLA de moins de 2 minutes. Le module M19 (Conformité) calcule un score de conformité réglementaire et génère les exports légaux.

9.1 Alertes DPMED - Flux critique (M18)

Le flux de diffusion d'une alerte DPMED est l'opération la plus critique de la plateforme. Le débit garanti de moins de 2 minutes nécessite une architecture optimisée. Le webhook DPMED est reçu, la signature RSA-256 et l'IP whitelist sont vérifiées, l'alerte est créée en base et mise en queue BullMQ avec priorité 1. Puis les pharmacies concernées (lots en stock actif) et les patients concernés (achats 90 derniers jours) sont identifiés. La diffusion se fait via push Firebase FCM (batch de 500 tokens) et SMS AfricasTalking (bulk). Enfin, les compteurs sont mis à jour et le portail DPMED est notifié via WebSocket.

9.2 Queue BullMQ dédiée

- Nom de la queue : 'alertes-dpmed'
- Priorité : 1 (maximum)
- Concurrency : 20 workers simultanés
- Retry : 5 tentatives avec backoff exponentiel (1s, 5s, 30s, 120s, 300s)
- Redis : instance Upstash dédiée (REDIS_ALERTES_URL)
- Monitoring : dashboard BullMQ Board sur /admin/queues

9.3 Score Conformité (M19)

Le score de conformité est calculé sur 100 points répartis en 5 composantes : registre stupefiants sans trou (25 points), alertes DPMED traitées en moins de 24h (25 points), documents valides non expirés (20 points), signalements EI soumis dans les délais (15 points), et PV de destructions à jour (15 points). Ce score est recalculé quotidiennement et affiché dans le tableau de bord de conformité. Les exports légaux génèrent les documents au format attendu par la DPMED et l'ABRP.

9.4 Étapes détaillées

Semaine	Action
S39-S40	Implementer le module M16 Pharmacovigilance : signalement EI, pseudonymisation, surveillance lots, interactions.
S40-S41	Implementer le module M18 Alertes DPMED : webhook, verification RSA-256, queue BullMQ, diffusion push+SMS.
S41-S42	Implementer les diffusions : Firebase FCM (batch 500 tokens), AfricasTalking SMS (bulk), WebSocket portail DPMED.
S42-S43	Implementer le module M19 Conformite : score sur 100 points, exports legaux, certification DPMED.
S43-S44	Implementer le portail institutionnel DPMED avec dashboard temps reel. Tests de performance SLA < 2 min.

PHASE 6 : INTEGRATIONS GROSSISTES ET SOBAPS

Durée estimée : Semaines 45-48 (4 semaines)

Les integrations avec les grossistes repartiteurs (UbiPharm, Promopharma) et la SoBAPS sont essentielles pour la chaine d'approvisionnement. UbiPharm et Promopharma fournissent des APIs pour passer des commandes, consulter les catalogues et confirmer les livraisons. La SoBAPS gere l'approvisionnement en medicaments essentiels generiques. Ces integrations utilisent des webhooks HMAC-SHA256 pour la securite et mTLS pour les connexions serveur a serveur.

10.1 Integrations Grossistes

- API UbiPharm : catalogue de produits, passage de commandes, confirmation de livraison, webhooks de statut
- API Promopharma : meme integration qu'UbiPharm avec endpoints dedies
- Portail grossiste : tableau de bord avec commandes recues, demande agreee, statistiques
- Webhooks HMAC-SHA256 : validation de chaque webhook entrant avec timingSafeEqual
- mTLS : certificats mutuels pour les connexions serveur a serveur avec les grossistes
- IP Whitelist : verification de l'IP source de chaque webhook institutionnel

10.2 Integration SoBAPS

- Portail SoBAPS : traçabilité des livraisons de medicaments essentiels generiques
- Endpoint POST /sobaps/receptions : confirmation de reception de livraison
- Webhook SoBAPS : notification de livraison en cours avec validation HMAC-SHA256
- Donnees isolees : schema PostgreSQL separe pour les donnees institutionnelles

10.3 Etapes detaillees

Semaine	Action
S45-S46	Implementer les modules M17 Grossistes : API UbiPharm + Promopharma, catalogues, commandes, webhooks.
S46-S47	Implementer le portail SoBAPS : traçabilité livraisons, confirmations, webhook. Schema PostgreSQL separe.
S47-S48	Implementer mTLS pour toutes les connexions serveur-institution. IP Whitelist sur les webhooks. Tests d'integration.

PHASE 7 : RESEAU NETWORK ET DEPLOIEMENT

Durée estimée : Semaines 49-52 (4 semaines)

Le reseau Network (MediHelm Network) est le portail pour les promoteurs qui gerent plusieurs officines. Il offre une vue d'ensemble du reseau, la gestion des officines, le stock reseau, et le personnel reseau. Le plan HELM NETWORK (sur devis) cible les reseaux de 2+ officines. Cette phase inclut aussi la mise en place de l'infrastructure de deploiement avec Docker, CI/CD, et monitoring.

11.1 MediHelm Network

- Dashboard reseau : KPIs consolides sur toutes les officines du reseau
- Gestion officines : ajout/suppression d'officines, transferts de stock inter-officines
- Stock reseau : vue globale du stock, alertes de peremption, suggestions de transferts
- Personnel reseau : planning consolide, conges, paie multi-officines
- JWT prometteur : contient { pharmacies: ['uuid1', 'uuid2', ...] } pour l'accès multi-tenant

11.2 Infrastructure de deploiement

- Docker : Dockerfile multi-stage pour apps/api et apps/web, docker-compose.yml pour le developpement local
- CI/CD : pipeline GitHub Actions (lint, test, build, deploy) avec stages de verification
- Deploiement : Vercel (frontend) + Railway (backend) avec auto-scaling
- Monitoring : Sentry pour les erreurs, Vercel Analytics pour les performances, BullMQ Board pour les queues
- SSL/TLS : certificats automatiques via Vercel et Railway, mTLS pour les connexions institutionnelles
- Sauvegarde : backups quotidiens PostgreSQL, retention 30 jours, test de restauration mensuel

11.3 Etapes detaillees

Semaine	Action
S49-S50	Implementer le portail Network : dashboard, gestion officines, stock reseau, personnel reseau, JWT multi-tenant.
S50-S51	Creer les Dockerfiles et docker-compose.yml. Configurer le CI/CD GitHub Actions.
S51-S52	Deployer sur Vercel + Railway. Configurer Sentry, Vercel Analytics, BullMQ Board. Tests de charge.

PHASE 8 : CERTIFICATION ET MISE EN PRODUCTION

Durée estimée : Semaines 53-56 (4 semaines)

La phase finale est dédiée aux tests de bout en bout, à la vérification de conformité complète avec la documentation, et à la mise en production progressive. Elle suit le séquencement de lancement documenté : Phase 1 Beta (mois 1-3, 5 officines pilotes gratuit), Phase 2 Beta payante (mois 4-6, -30%), Phase 3 Plein tarif (mois 7+). La certification implique la vérification de chaque règle absolue, chaque endpoint critique, et chaque flux métier documenté.

12.1 Checklist de conformité

- Chaque requête Prisma filtre par pharmacieId (multitenancy strict)
- TenantMiddleware injecte SET app.current_tenant avant chaque requête
- RLS PostgreSQL actif sur toutes les tables métier
- Tous les webhooks institutionnels valides par HMAC-SHA256
- Alertes DPMED validées par RSA-256 avant mise en queue
- mTLS pour les connexions serveur-institutions
- Zero donnée individuelle d'officine vers les portails institutionnels
- Mode offline fonctionnel : POS et Stock hors connexion
- Champ synchedAt fonctionnel sur Vente
- FEFO implémenté pour le décrement stock
- CMUP recalcule à chaque réception de lot
- Score conformité sur 100 points avec les 5 composantes documentées
- SLA alertes DPMED inférieur à 2 minutes
- Rate limiting sur /auth/login (5/15min/IP)
- Clefs API chiffrées en base (AES-256-GCM)
- Pseudonymisation des signalements EI avant transmission DPMED

12.2 Plan de lancement

Phase	Periode	Prix	Objectif
Beta gratuite	Mois 1-3	Gratuit	5 officines pilotes à Cotonou et Parakou
Beta payante	Mois 4-6	-30% (SEED 13 900, GROW 24 500, LEAD 38 500 FCFA)	20-30 officines, feedback iteration
Plein tarif	Mois 7+	Plein tarif (des partenariats institutionnel actif)	100+ officines, partnerships actifs

13. SCHEMA PRISMA : CHANGEMENTS REQUIS

Le schema Prisma actuel contient 77 modeles et 31 enums, ce qui couvre l'ensemble des 19 modules. Cependant, plusieurs champs et enums doivent etre modifies, ajoutees ou renommes pour etre conformes a la documentation. Ces changements sont prerequis pour les phases 0 (migration monorepo), 1 (auth), 2 (offline), et 5 (alertes DPMED).

13.1 Champs a ajouter

Modele	Champ	Type	Contrainte	Justification
Vente	reference	String	@unique	Idempotence sync offline, reference unique
Vente	synchedAt	DateTime?	-	Null = cree offline, date = synchronise
Vente	montantAssur	Float	@default(0)	Montant couvert par l'assurance
Vente	commandePatId	String?	-	Lien vers commande patient en ligne
Pharmacie	slug	String	@unique	URL slug pour les pages publiques
Pharmacie	modeGardeActif	Boolean	@default(false)	Indicateur pharmacie de garde active
Pharmacie	planExpireAt	DateTime?	-	Date d'expiration du plan tarifaire
Utilisateur	supabaseUid	String	@unique	Identifiant Supabase Auth

13.2 Enums a modifier

Enum	Changement	Detail
PlanType	STEM devient SEED	La documentation specifie HELM SEED, pas HELM STEM
StatutVente	Ajouter AVOIR	Avoir/retour sur vente, necessaire pour les retours
StatutVente	Ajouter PARTIELLEMENT_PAYEE	Vente partiellement payee (credit patient)
ModePaiement	Ajouter ASSURANCE	Paiement par assurance/tiers payant
ModePaiement	Ajouter CREDIT	Paiement a credit (credit patient)
StatutCommandePatient	Passer en francais	RECUE, EN_PREPARATION, PRETE, RECUPEREE, ANNULEE

Enum	Changement	Detail
Nouveau : DomaineIA	Creer l'enum	STOCK, VENTES, PATIENTELE, PERSONNEL, FINANCE, PEREMPTIONS, RESEAU, CONFORMITE, INTEGRATION_GROSSISTE
RapportAnalytics	domaine : DomaineIA	Remplacer le champ String par l'enum DomaineIA

13.3 RLS PostgreSQL

Row Level Security doit etre active sur toutes les tables metier. Chaque table doit avoir une politique qui filtre par pharmacieId en utilisant la variable de session app.current_tenant positionnee par le TenantMiddleware. Les tables institutionnelles (AlerteDPMED, ScoreConformite, etc.) sont dans un schema PostgreSQL separe sans RLS mais avec un acces restreint par role. La mise en place du RLS est prerequisite pour la Phase 1 et doit etre testee avec des utilisateurs de differentes pharmacies pour verifier l'isolation.

14. VARIABLES D'ENVIRONNEMENT MANQUANTES

La documentation specifie un ensemble complet de variables d'environnement avec des noms officiels qui ne doivent jamais etre modifies. Actuellement, seules 12 variables sont referencees dans le code sur les 40+ documentees. Voici les variables manquantes organisees par categorie.

14.1 Variables critique manquantes

Variable	Usage	Phase
SUPABASE_URL	Connexion Supabase Auth	Phase 1
SUPABASE_ANON_KEY	Cle publique Supabase	Phase 1
SUPABASE_SERVICE_KEY	Cle service Supabase (cote serveur)	Phase 1
JWT_SECRET	Secret JWT access token (15 min)	Phase 1
JWT_REFRESH_SECRET	Secret JWT refresh token (7 jours)	Phase 1
REDIS_URL	Connexion Redis Upstash (sessions, cache)	Phase 1
REDIS_ALERTES_URL	Redis dedie pour la queue alertes DPMED	Phase 5
SENTRY_DSN	Monitoring d'erreurs Sentry	Phase 7
FIREBASE_SERVER_KEY	Cle serveur Firebase FCM pour push	Phase 5

14.2 Variables integration manquantes

Variable	Usage	Phase
FEDAPAY_API_KEY	Cle API Fedapay (paiements)	Phase 4
AFRICAS_TALKING_KEY	Cle API AfricasTalking (SMS)	Phase 5
AFRICAS_TALKING_USERNAME	Username AfricasTalking	Phase 5
RESEND_API_KEY	Cle API Resend (emails)	Phase 4
DPMED_PUBLIC_KEY	Cle publique RSA DPMED (verification signature)	Phase 5
DPMED_API_URL	URL API DPMED	Phase 5
DPMED_IP_WHITELIST	IPs autorisees pour webhooks DPMED	Phase 5
SOBAPS_API_URL	URL API SoBAPS	Phase 6
SOBAPS_IP_WHITELIST	IPs autorisees pour webhooks SoBAPS	Phase 6
MTLS_CERT_PATH	Chemin certificat mTLS	Phase 6

Variable	Usage	Phase
MTLS_KEY_PATH	Chemin cle privée mTLS	Phase 6
MTLS_CA_PATH	Chemin autorite de certification mTLS	Phase 6

15. RISQUES ET MITIGATION

Risque	Probabilite	Impact	Mitigation
Migration monorepo casse le frontend existant	Moyen	Eleve	Migrer incrementalement avec feature flags. Garder l'ancien build fonctionnel pendant la transition.
Supabase Auth incompatible avec le flow NextAuth	Moyen	Eleve	Implementer un adapter NextAuth-Supabase. Migrer les utilisateurs existants avec script de migration.
Mode offline ne sync pas correctement	Eleve	Critique	Tests exhaustifs de scenarios de conflit. Mecanisme de rollback. Surveillance des sync errors en production.
SLA alertes DPMED non respecte	Moyen	Critique	Load testing avec 100+ pharmacies. Queue dediee Redis. Monitoring du temps de bout en bout.
Integrations grossistes indisponibles	Eleve	Moyen	Mode degraded avec catalogue local. Retry automatique. Cache des catalogues en Redis.
Donnees perdues pendant migration Prisma	Faible	Critique	Backup complet avant migration. Migration sur base de test d'abord. Rollback automatique.
Performance degradee avec RLS	Moyen	Moyen	Index composites sur pharmacieId. Benchmark avant/apres RLS. Optimisation des queries.
Equipe insuffisante pour 56 semaines	Eleve	Eleve	Prioriser les phases 0-2 (securite + offline). Les phases 4-7 peuvent etre repoussees. Recrutement.

16. CALENDRIER CONSOLIDE

Phase	Semaines	Duree	Livrable principal
Phase 0	S1-S6	6 sem.	Monorepo + NestJS + schema Prisma corrige
Phase 1	S7-S12	6 sem.	Auth Supabase + RBAC 100% + Zod + rate limiting
Phase 2	S13-S20	8 sem.	Mode offline + SQLite + sync + PWA + FEFO
Phase 3	S21-S32	12 sem.	Modules M01-M09 complets dans NestJS
Phase 4	S33-S38	6 sem.	Espace Patient + Analytics IA + Fedapay reel
Phase 5	S39-S44	6 sem.	Pharmacovigilance + Alertes DPMED + Conformite
Phase 6	S45-S48	4 sem.	Grossistes + SoBAPS + mTLS
Phase 7	S49-S52	4 sem.	Network + Docker + CI/CD + monitoring
Phase 8	S53-S56	4 sem.	Certification + beta 5 officines
TOTAL	S1-S56	56 sem.	Plateforme 100% conforme a la documentation

Ce calendrier represente l'estimation la plus realiste pour atteindre une conformite complete avec la documentation MediHelm. Les phases 0 a 2 sont non negociables et constituent le socle minimal pour une mise en production securisee. Les phases 3 a 5 implementent les modules metier et institutionnels qui font la valeur ajoutee de MediHelm. Les phases 6 a 8 complètent l'ecosysteme avec les integrations externes, le reseau Network, et la certification finale. En cas de contrainte de temps, les phases peuvent etre partiellement parallelisees, mais les dependances doivent etre respectees pour garantir la coherence du systeme.